Laboratório 04:

Problemas de Satisfação de Restrições (CSP)

CC7711 - Inteligência Artificial

Prof. Dr. Murillo | Depto. de Ciência da Computação

O que define um CSP?

Modelar problemas como CSP exige definir 3 pilares fundamentais de representação de estado:

- **Variáveis (X):** Os elementos que precisam receber uma atribuição (ex: Cor de um estado, valor de uma célula).
- **Domínios (D):** O conjunto de valores legais que podem ser associados a cada variável respectiva.
- **Restrições (C):** As regras lógicas que especificam as combinações permitidas (o que é proibido pela topologia do problema).

> **Objetivo Central:** Alcançar uma atribuição que seja ao mesmo tempo *completa* (todas as variáveis têm valores) e *consistente* (nenhuma restrição é violada).

O que define um CSP?

```python
class CSP:
    def __init__(self, variables, domains, neighbors):
        """
        Inicializa o CSP com suas variáveis, domínios permitidos e o
        grafo de vizinhança (restrições).
        """
        self.variables = variables
        self.domains = domains
        self.neighbors = neighbors


        self.nodes_expanded = 0
        self.backtracks = 0
```

Desafio 1: A Região Sudeste do Brasil

Vamos colorir o mapa físico do Sudeste utilizando apenas 3 cores. Como estruturamos este problema?

- **Variáveis (X):** Os quatro estados do sudeste (SP, RJ, MG, ES).
- **Domínios (D):** ['Vermelho', 'Verde', 'Azul']. (3 cores bastam para essa topologia).

> **A Restrição de Ouro:** Estados que compartilham uma fronteira física devem obrigatoriamente possuir cores diferentes.

Desafio 1 – Implementando o Grafo da Região Sudeste

O primeiro passo é representar as fronteiras do mapa como um dicionário de adjacências na sua IDE, acompanhado de uma função avaliadora simples.

```python
def create_map_coloring_csp():
    """
    Modelagem topológica da Região Sudeste do Brasil.
    SP, RJ e MG formam um triângulo (requerem 3 cores).
    ES faz fronteira apenas com RJ e MG.
    """
    variables = ['SP', 'RJ', 'MG', 'ES']

    # 3 Cores são perfeitamente suficientes para este subgrafo
    domains = {var: ['Vermelho', 'Verde', 'Azul'] for var in variables}

    neighbors = {
        'SP': ['RJ', 'MG'],
        'RJ': ['SP', 'MG', 'ES'],
        'MG': ['SP', 'RJ', 'ES'],
        'ES': ['RJ', 'MG']
    }
    return CSP(variables, domains, neighbors)
```

Desafio 2: Sudoku como CSP

Elevamos a complexidade. Como mapear o Sudoku?

- **Estrutura:** 81 variáveis (uma para cada célula do tabuleiro).
- **Domínio Fixo:** Numérico de {1, 2, 3, 4, 5, 6, 7, 8, 9}.
- **Restrições Universais:**
  - Linhas não podem conter valores repetidos.
  - Colunas não podem conter valores repetidos.
  - Subgrades 3x3 devem conter valores únicos.

			2	6		7		1
6	8			7			9	
1	9				4	5		
8	2		1				4	
		4	6		2	9		
	5				3		2	8
		9	3				7	4
	4			5			3	6
7		3		1	8			

Desafio 2: Sudoku como CSP

```python
def create_sudoku_csp(grid):
    variables = list(range(81))
    domains = {}
    neighbors = {var: set() for var in variables}

    # Inicialização dos domínios (1 a 9 para vazios, valor fixo para preenchidos)
    for i in variables:
        if grid[i] == 0:
            domains[i] = [1, 2, 3, 4, 5, 6, 7, 8, 9]
        else:
            domains[i] = [grid[i]]

    #Completar o calculo das vizinhanças e a regra do Sudoku:
```

O Algoritmo Base: Backtracking

Uma busca em profundidade pura que "desfaz a jogada" (retrocesso) imediatamente ao detectar que uma restrição foi violada.

```python
def backtracking_search(self, use_mrv=False):
    """Inicia a busca e contabiliza o tempo de execução."""
    self.nodes_expanded = 0
    self.backtracks = 0

    start_time = time.time()
    solution = self._backtrack({}, use_mrv)
    end_time = time.time()

    return {
        'solution': solution,
        'time': end_time - start_time,
        'nodes_expanded': self.nodes_expanded,
        'backtracks': self.backtracks
    }
```

Criar dentro da classe CSP

O Algoritmo Base: Backtracking

```python
def _backtrack(self, assignment, use_mrv):
    """O motor de busca em profundidade com recursão e retrocesso."""
    # Condição de parada: atribuição completa
    if len(assignment) == len(self.variables):
        return assignment

    # Seleção de Variável
    if use_mrv:
        var = self.mrv(assignment)
    else:
        var = [v for v in self.variables if v not in assignment][0]

    self.nodes_expanded += 1

    for value in self.domains[var]:
        if self.is_consistent(var, value, assignment):
            assignment[var] = value

            result = self._backtrack(assignment, use_mrv)
            if result:
                return result

            # O charme do Backtracking: desfazer a ação se der errado
            del assignment[var]
            self.backtracks += 1

    return None
```

Criar dentro da classe CSP

A Inteligência da Heurística MRV

Rodar o Sudoku apenas com backtracking puro causará uma explosão combinatória severa. A solução está em ordenar a escolha das variáveis.

- **A Explosão:** O espaço de estados do Sudoku é massivo. Tentativas cegas demoram excessivamente.
- **A Solução (MRV):** A heurística *Minimum Remaining Values* (Valores Restantes Mínimos).
- **A Lógica:** O algoritmo escolhe sempre, em cada passo, a variável que possui o menor número de opções legais restantes em seu domínio.

> **O Princípio "Falhe Rápido":** Ao forçar as variáveis mais restritas primeiro, podamos ramos inviáveis da árvore de busca no início do processamento.

A Inteligência da Heurística MRV

```python
def mrv(self, assignment):
    """
    Heurística Minimum Remaining Values (MRV).
    """
    unassigned = [v for v in self.variables if v not in assignment]

    def count_valid_values(var):
        count = 0
        for val in self.domains[var]:
            if self.is_consistent(var, val, assignment):
                count += 1
        return count

    return min(unassigned, key=count_valid_values)
```

Roteiro do Laboratório & Entregas

Utilizem o template de código Python (.ipynb) fornecido para completar as seguintes missões:

- **Passo 1:** Implementar a modelagem CSP para resolver a Coloração de Mapas, restrita estritamente à topologia da Região Sudeste do Brasil.
- **Passo 2:** Implementar o motor CSP para resolver o tabuleiro de Sudoku, inserindo a heurística MRV obrigatória para viabilizar o processamento.

Métricas e Submissão: Submetam o notebook com o código completo. Ao final, apresentem a coloração de mapas da região sudeste (pode ser um grafo) e uma comparação entre os cenários (Sudoku com busca cega e MRV) com as métricas: **Tempo Total (s)**, **Nós Expandidos** e **Backtracks Realizados**.

Casos de teste Sudoku – Iniciante

```
sudoku_iniciante = [
    5, 3, 0, 0, 7, 0, 0, 0, 0,
    6, 0, 0, 1, 9, 5, 0, 0, 0,
    0, 9, 8, 0, 0, 0, 0, 6, 0,

    8, 0, 0, 0, 6, 0, 0, 0, 3,
    4, 0, 0, 8, 0, 3, 0, 0, 1,
    7, 0, 0, 0, 2, 0, 0, 0, 6,

    0, 6, 0, 0, 0, 0, 2, 8, 0,
    0, 0, 0, 4, 1, 9, 0, 0, 5,
    0, 0, 0, 0, 8, 0, 0, 7, 9
]
```

```
= Solução do Sudoku =
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
- - - - - - - - - - -
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
- - - - - - - - - - -
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
=====================
```

Casos de teste Sudoku – Intermediário

```
sudoku_intermediario = [
    0, 0, 5,  3, 0, 0,  0, 0, 0,
    8, 0, 0,  0, 0, 0,  0, 2, 0,
    0, 7, 0,  0, 1, 0,  5, 0, 0,
[redacted]
    4, 0, 0,  0, 0, 5,  3, 0, 0,
    0, 1, 0,  0, 7, 0,  0, 0, 6,
    0, 0, 3,  2, 0, 0,  0, 8, 0,
[redacted]
    0, 6, 0,  5, 0, 0,  0, 0, 9,
    0, 0, 4,  0, 0, 0,  0, 3, 0,
    0, 0, 0,  0, 0, 9,  7, 0, 0
]
```

```
= Solução do Sudoku =
1 4 5 | 3 2 7 | 6 9 8
8 3 9 | 6 5 4 | 1 2 7
6 7 2 | 9 1 8 | 5 4 3
- - - - - - - - - - -
4 9 6 | 1 8 5 | 3 7 2
2 1 8 | 4 7 3 | 9 5 6
7 5 3 | 2 9 6 | 4 8 1
- - - - - - - - - - -
3 6 7 | 5 4 2 | 8 1 9
9 8 4 | 7 6 1 | 2 3 5
5 2 1 | 8 3 9 | 7 6 4
=====================
```

Casos de teste Sudoku – Intermediário

```
sudoku_hard = [
    8, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 3, 6, 0, 0, 0, 0, 0,
    0, 7, 0, 0, 9, 0, 2, 0, 0,
    0, 5, 0, 0, 0, 7, 0, 0, 0,
    0, 0, 0, 0, 4, 5, 7, 0, 0,
    0, 0, 0, 1, 0, 0, 0, 3, 0,
    0, 0, 1, 0, 0, 0, 0, 6, 8,
    0, 0, 8, 5, 0, 0, 0, 1, 0,
    0, 9, 0, 0, 0, 0, 4, 0, 0
]
```

```
= Solução do Sudoku =
8 1 2 | 7 5 3 | 6 4 9
9 4 3 | 6 8 2 | 1 7 5
6 7 5 | 4 9 1 | 2 8 3
- - - - - - - - - - -
1 5 4 | 2 3 7 | 8 9 6
3 6 9 | 8 4 5 | 7 2 1
2 8 7 | 1 6 9 | 5 3 4
- - - - - - - - - - -
5 2 1 | 9 7 4 | 3 6 8
4 3 8 | 5 2 6 | 9 1 7
7 9 6 | 3 1 8 | 4 5 2
=====================
```